

PROGRESSION · TIMELINE · GRAPH

state-chart

Native state machine diagram — states as a numbered list, transitions as nested inline-code refs.

SUBMISSION LIFECYCLE

States connect; the arrows carry the rules.

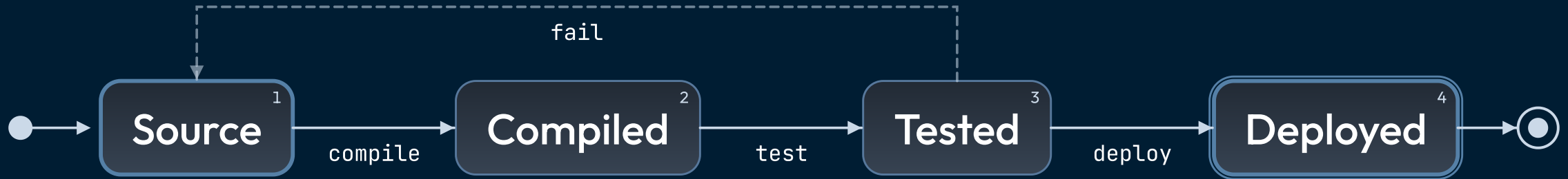
How a draft moves from author to publication.



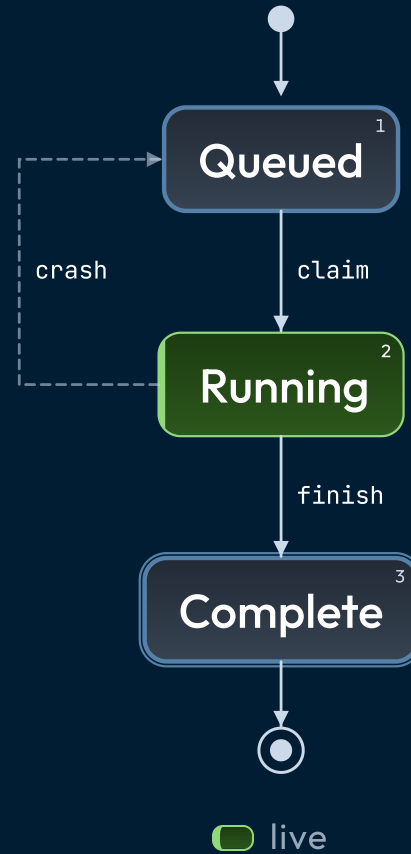
Rejected drafts return to the author; revisions stay in review.

■ on-track ■ at-risk

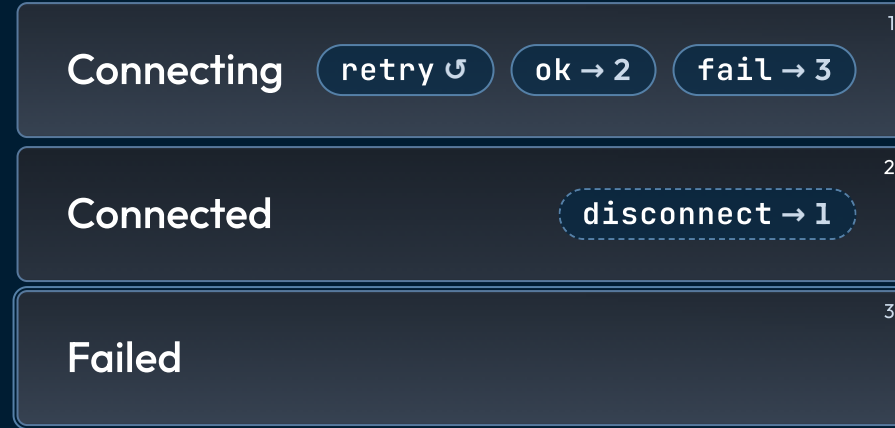
lr flows the states left to right.



tb stacks the states top to bottom.

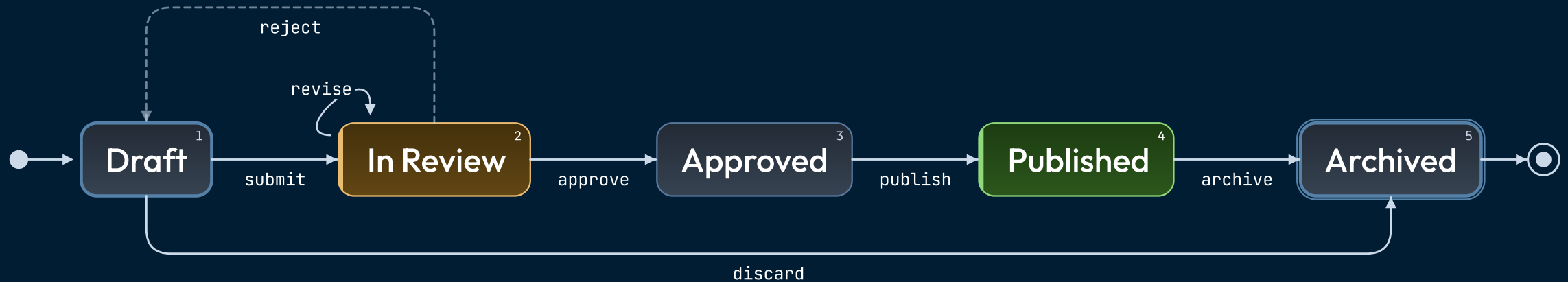


inline sets the chart beside its prose.



 live

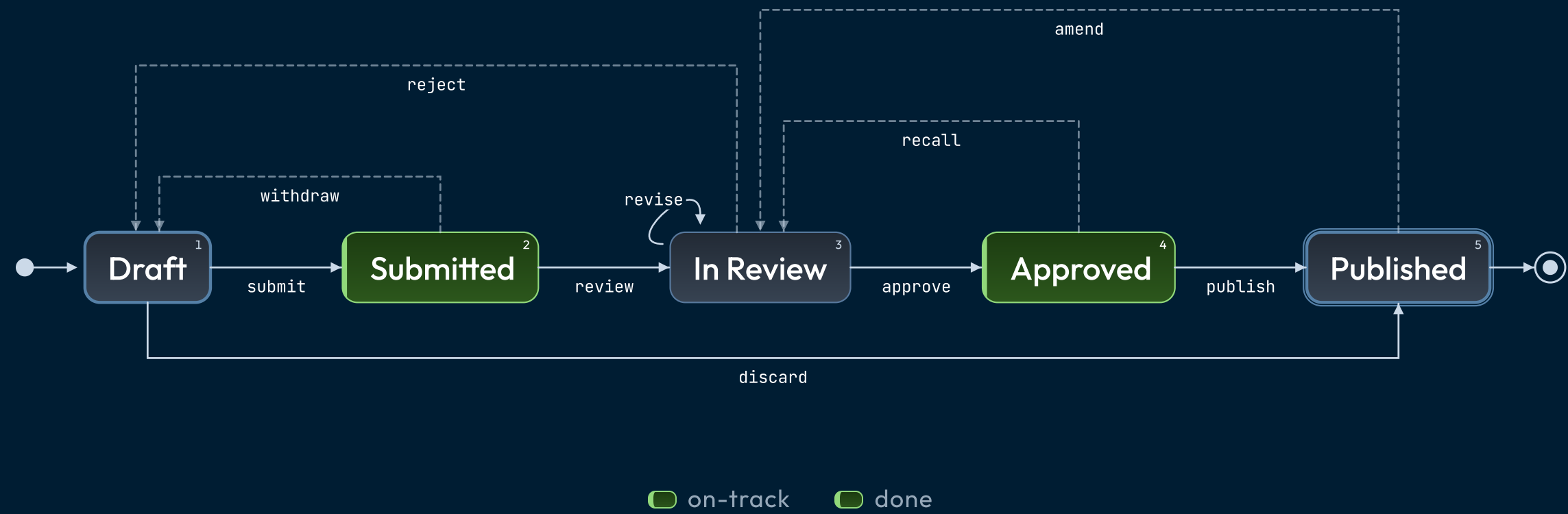
curved eases the arrows between states.



at-risk live

SUBMISSION LIFECYCLE

Stress test — back-edges, skips, self-loop.



SUBMISSION LIFECYCLE

States connect; the arrows carry the rules.

How a draft moves from author to publication.



Rejected drafts return to the author; revisions stay in review.

■ on-track ■ at-risk

SUBMISSION LIFECYCLE

States connect; the arrows carry the rules.

How a draft moves from author to publication.



Rejected drafts return to the author; revisions stay in review.

■ on-track ■ at-risk

SUBMISSION LIFECYCLE

States connect; the arrows carry the rules.

How a draft moves from author to publication.



Rejected drafts return to the author; revisions stay in review.

■ on-track ■ at-risk

When NOT to reach for state-chart.

More than ~12 states

A long chain wraps onto more lines rather than shrinking, so eight or ten states still read. Past about a dozen the machine stops reading as a machine and starts reading as a list, however it is laid out. Group the states into phases and show one phase at a time, or step back to a higher-level abstraction. The chart's job is to make the topology obvious in one glance.

Hierarchical or parallel states

v1 grammar is one flat list of states with one outgoing arrow per nested bullet. Composite states, orthogonal regions, history nodes — anything Mermaid's `stateDiagram-v2` does and this layout doesn't — belong in a Mermaid fence via the `diagram` component.

Continuous processes

If the diagram is really a workflow with stages that overlap or block (queue depth, throughput, capacity), a `gantt` or `kanban` chart reads better. State charts are for discrete, mutually-exclusive states the system flips between.

See also.

`diagram` — the machine has hierarchical states, parallel regions, or guards that need Mermaid's full state-diagram grammar

`journey` — the sequence is a user's path through tasks with mood / affect, not a system's discrete states

`timeline-list` — events are points in time rather than transitions between named states

`list-steps` — a linear procedure with no branching — state-chart is overkill if there are no choices to make

`roadmap` — parallel workstreams across phases, not a single machine's transitions